

Домашнее задание: RAG и протоколы взаимодействия ИИ-агентов

Курс: Промышленная разработка ИИ-агентов Темы: RAG и протоколы взаимодействия ИИ-агентов

Цель задания

Объединить две темы курса в одном практическом проекте:

1. RAG: подготовить корпус документов, разбить его на фрагменты, сохранить метаданные, построить индекс и проверить качество поиска.
2. Протоколы взаимодействия ИИ-агентов: подключить поиск как внешний инструмент через готовый MCP-сервер и показать, что ИИ-клиент может обращаться к базе знаний через стандартную протокольную границу.

Итоговая схема:

документы → фрагменты → метаданные → индекс → MCP-сервер/инструмент → top-k результатов поиска

В этой работе оценивается связка поиск в RAG + MCP-интерфейс + агент: насколько воспроизводимо подготовлен индекс, насколько корректно возвращаются источники и насколько понятно ИИ-агент может пользоваться поиском через MCP. Генерация развёрнутого ответа поверх найденных фрагментов необязательна, но агент должен уметь вызвать MCP-инструмент и вернуть найденные top-k фрагменты.

Задание

Подготовьте базу знаний и подключите её к ИИ-агенту через готовый MCP-сервер для базы знаний, поискового сервиса или векторного хранилища. Подготовьте собственный корпус документов, проиндексируйте его и продемонстрируйте поиск через агента: по заданному запросу агент вызывает MCP-инструмент и возвращает top-k фрагментов с указанием источника — идентификатора документа и идентификатора фрагмента.

Готовый MCP-сервер не обязательно должен выполнять всю подготовку корпуса. Если выбранный сервер поддерживает только поиск, подготовку корпуса и индексацию можно выполнить через SDK, CLI или API выбранного хранилища. Но демонстрация поиска должна идти именно через MCP-инструмент.

Требования

1. Выберите готовый MCP-сервер

Можно использовать MCP-сервер для:

- векторной базы данных;
- управляемой базы знаний;
- поискового движка;
- локального или облачного провайдера поиска;
- другой системы, которая умеет возвращать релевантные фрагменты по запросу.

Не нужно писать MCP-сервер с нуля. Важно уметь объяснить, какой MCP-инструмент вызывается и какие данные он возвращает.

Примеры бесплатных вариантов

Нижеследующий — не обязательный список, а подсказки, с чего можно начать. Можно выбрать другой готовый MCP-сервер, если он подходит под требования задания.

Вариант	Почему подходит	На что обратить внимание
Chroma / ChromaDB	Самый простой локальный вариант для учебного RAG: можно хранить фрагменты и эмбединги на диске без облака.	У Chroma часто встречаются community MCP-серверы. Проверьте, что выбранный сервер действительно возвращает document_id, chunk_id и source.
Qdrant	Есть официальный MCP-сервер mcp-server-qdrant; сам Qdrant можно запустить локально или использовать in-memory/локальный режим для демо.	Хороший вариант по умолчанию, если хочется меньше спорить с инфраструктурой. Индексацию можно сделать отдельным скриптом, а поиск показать через MCP.
Milvus	Open-source векторная база; есть MCP-сервер для поиска и операций с коллекциями.	Чуть тяжелее для первого запуска, зато ближе к промышленному варианту. Для локального демо можно посмотреть Milvus Lite или локальный контейнер.
Weaviate	Open-source векторная база с гибридным поиском; есть готовые MCP-интеграции.	Может потребовать больше настройки схемы и сервера, чем Chroma/Qdrant.
Локальный поисковый сервер поверх файлов	Подходит, если вы нашли готовый MCP-сервер, который индексирует папку с markdown/txt/pdf и умеет искать по ней.	Убедитесь, что это не просто файловый поиск: должны быть фрагменты, метаданные и top-k результаты.

Облачные бесплатные тарифы использовать можно, но осторожно: не кладите ключи в репозиторий, добавьте .env.example и опишите, как получить и заполнить переменные окружения.

2. Подготовьте корпус документов

Минимальный объём:

- 5–10 документов; или
- один большой документ, разбитый на достаточное количество смысловых фрагментов; или
- эквивалентный текстовый корпус по выбранной теме.

Для каждого фрагмента сохраните метаданные:

- document_id — идентификатор документа;
- chunk_id — идентификатор фрагмента;
- source — путь, URL или другое описание источника;

- желательно: title, section, page, created_at или другой полезный указатель.

3. Выполните разбиение и индексацию

Нужно показать, как документы превращаются в индекс:

1. загрузка исходных документов;
2. разбиение на фрагменты;
3. добавление метаданных;
4. загрузка фрагментов в выбранное хранилище;
5. проверка, что метаданные возвращаются вместе с результатами поиска.

4. Подключите MCP-сервер

В работе должны быть:

- пример конфигурации MCP;
- инструкция, как запустить сервер;
- название инструмента, который выполняет поиск;
- один пример ручного вызова инструмента или журнал его работы.

5. Подключите MCP-инструмент к агенту

Поиск должен быть доступен не только как ручной вызов MCP-инструмента, но и как инструмент агента. Агент может быть реализован на одном из фреймворков:

- LangChain;
- LangGraph;
- Deep Agents.

Для LangChain и LangGraph удобно использовать библиотеку langchain-mcp-adapters: она превращает MCP-инструменты в LangChain tools и позволяет подключать один или несколько MCP-серверов через MultiServerMCPClient.

Минимальный ориентир:

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient({
    "knowledge_base": {
        "transport": "stdio",
        "command": "...",
        "args": ["..."],
    }
})

tools = await client.get_tools()
agent = create_agent(model, tools)
result = await agent.ainvoke({
    "messages": "Найди top-3 фрагмента про риски применения ИИ"
})
```

Если используете Deep Agents, можно подключить те же MCP-инструменты как tools агента или явно описать промежуточный слой, который вызывает MCP и отдаёт результаты агенту. В отчёте важно показать: агент получил запрос, выбрал инструмент поиска, вызвал его и вернул фрагменты с document_id и chunk_id.

6. Продемонстрируйте top-k поиск

Поиск должен принимать:

```
query: текстовый запрос
k: количество результатов
```

Результат должен содержать для каждого найденного фрагмента:

- текст фрагмента или его краткий отрывок;
- document_id;
- chunk_id;
- source;
- оценку релевантности, расстояние или ранг, если MCP-инструмент это возвращает.

Минимальный пример вывода:

```
Query: Какие риски описаны в документе?
Top-k: 3

1. document_id=doc_01, chunk_id=doc_01_chunk_04, score=0.82
   source=docs/risk_policy.md
   text=...

2. document_id=doc_03, chunk_id=doc_03_chunk_02, score=0.76
   source=docs/ai_policy.md
   text=...
```

7. Проведите проверку на 15–20 запросах

Подготовьте набор из 15–20 запросов. Запросы должны покрывать разные случаи:

- точный факт;
- терминологический вопрос;
- вопрос по конкретному документу;
- широкий смысловой вопрос;
- вопрос, на который в корпусе нет хорошего ответа;
- вопрос с несколькими релевантными источниками;
- вопрос, где важны метаданные или источник.

Для каждого запроса зафиксируйте результаты и краткую ручную оценку.

Рекомендуемый формат таблицы:

№	Запрос	Ожидаемый источник	Первый найденный фрагмент	Попал в top-3	Комментарий
1	Что такое ...?	doc_03	doc_03/chunk_12	да	точное попадание
2	Какие риски ...?	doc_01, doc_04	doc_04/chunk_02	да	ответ распределён по нескольким источникам

№	Запрос	Ожидаемый источник	Первый найденный фрагмент	Попал в top-3	Комментарий
3	Кто автор ...?	нет ответа	doc_07/chunk_01	нет	вопрос вне корпуса

Минимальные поля для отчёта:

```
query
top_k_results
document_id
chunk_id
score или rank
manual_judgement
comment
```

Названия полей можно оставить на английском, если они совпадают с полями в коде или API.

8. Приложите инструкцию запуска

Инструкция должна позволять другому человеку воспроизвести результат.

Опишите:

1. как установить зависимости;
2. какие переменные окружения нужны;
3. как подготовить или скачать корпус;
4. как выполнить разбиение и индексацию;
5. как запустить MCP-сервер;
6. как выполнить демонстрационный запрос;
7. как запустить проверку на 15–20 запросах.

Что сдавать

Рекомендуемая структура сдачи:

```
README.md
mcp_config.json / пример конфигурации MCP
ingest.py или ingest.ipynb
agent_demo.py или agent_demo.ipynb
search_demo.py или search_demo.ipynb
eval_queries.csv или eval_report.md
requirements.txt или pyproject.toml
```

Если используете облачную базу или закрытый провайдер, не прикладывайте секреты. Вместо этого добавьте `.env.example` и опишите, какие переменные нужно заполнить.

Критерии оценки

Критерий	Вес
Корпус для RAG подготовлен: разбиение, метаданные, индексация	20%

Критерий	Вес
Подключён готовый MCP-сервер и понятно, какой инструмент используется	20%
MCP-инструмент подключён к агенту на LangChain, LangGraph или Deep Agents	20%
Агент возвращает top-k с document_id, chunk_id и метаданными источника	15%
Есть проверка на 15–20 запросах и краткая ручная оценка качества поиска	15%
Инструкция запуска воспроизводима	10%

Что будет считаться хорошим результатом

Хорошая работа показывает не только «оно запускается», но и почему результату можно доверять:

- фрагменты имеют стабильные идентификаторы;
- метаданные не теряются при индексации;
- поиск можно воспроизвести через MCP-инструмент и через агента;
- в отчёте видно, где поиск сработал хорошо, а где не нашёл подходящее подтверждение;
- инструкция запуска не требует догадок.

Частые ошибки

1. Индекс создан, но метаданные не возвращаются в результатах поиска.
2. Поиск показан через SDK, но не через MCP-инструмент.
3. Нет стабильного chunk_id, поэтому невозможно сослаться на фрагмент.
4. В проверке есть только 2–3 запроса вместо 15–20.
5. Нет отрицательных запросов, на которые в корпусе нет ответа.
6. README не объясняет, как повторить подготовку корпуса и запуск MCP-сервера.
7. В репозиторий случайно попали API-ключи или учётные данные.

Дополнительное задание по желанию

Можно добавить простую генерацию ответа поверх найденных фрагментов:

```
query → MCP search → top-k фрагментов → ответ LLM со ссылками на источники
```

Но это необязательная часть. Основное задание — поиск через MCP, подключение MCP-инструмента к агенту и проверка качества поиска.